

Accelerate Avians

Complete Project Documentation

A Real-Time Multiplayer Typing Racer
with Retro Pixel-Art Birds

Next.js 16 + React 19 | Express + Socket.IO | PostgreSQL + Prisma | Clerk Auth

Frontend: Vercel | Game Server: Railway

Table of Contents

1.	Project Overview	3
2.	Tech Stack & Architecture	3
3.	Database Schema	4
4.	Authentication Flow	6
5.	Frontend Pages & Routes	7
6.	API Routes	9
7.	The Typing Engine	11
8.	Ghost Replay System	12
9.	Canvas Rendering	13
10.	Multiplayer Architecture	14
11.	React Hooks	16
12.	Rate Limiting & Anti-Cheat	17
13.	Seed Script & Bot System	18
14.	Key Components	19
15.	Middleware	20
16.	Environment Variables & Config	21
17.	Testing	21
18.	Deployment	22
19.	File Reference Map	22

1. Project Overview

Accelerate Avians is a real-time multiplayer typing racer with a retro pixel-art bird theme. Players type passages of text as quickly and accurately as possible, and their progress is visualized as a bird flying across a scrolling 2D canvas. The game supports two modes:

Solo Mode

The player races against ghost replays. Ghosts include the player's personal best on that passage (if they have one) plus up to 3 bot opponents matched to the player's skill level. Bots are pre-seeded in the database at various WPM levels (20, 25, 30, 40, 55, 70, 85, 100) with pre-generated ghost data so they feel like real opponents.

Multiplayer Mode

Up to 6 players join a room via a unique room code. The host selects a difficulty and starts the race. All players see each other's birds moving in real time via WebSocket communication. Final rankings are calculated server-side to prevent cheating.

The application features user authentication, persistent stats and race history, a global leaderboard, customizable bird avatars (19 bird species), and a CRT scanline visual effect for retro aesthetics.

2. Tech Stack & Architecture

High-Level Architecture

The project is split into two independently deployed services that share a single PostgreSQL database:

1. Next.js Frontend + API (deployed on Vercel)

Handles all page rendering (server and client components), REST API routes for scores, leaderboards, profiles, and passages, and serves the static assets (bird sprites, fonts, styles). Built with Next.js 16 and React 19.

2. Express Game Server (deployed on Railway)

A standalone Node.js server running Express and Socket.IO. Handles all real-time multiplayer communication: room creation, player joining, live typing progress broadcasting, race state management, and result calculation. This CANNOT run on Vercel because Vercel's serverless functions don't support persistent WebSocket connections.

Dependencies Breakdown

Runtime Dependencies

- next 16.2.1 - React framework for the frontend
- react 19.2.4 - UI library
- @clerk/nextjs ^7.0.7 - Authentication (sign-in, sign-up, session management)
- @prisma/client ^7.6.0 - Database ORM for PostgreSQL
- socket.io-client ^4.8.3 - WebSocket client for multiplayer
- @sentry/nextjs ^10.47.0 - Error monitoring and performance tracking
- svix ^1.89.0 - Webhook signature verification (for Clerk webhooks)

Dev Dependencies

- vitest ^4.1.2 - Unit testing framework

- playwright ^1.58.2 - End-to-end browser testing
- tailwindcss ^4 - Utility-first CSS framework
- typescript ^5 - Type safety
- eslint ^9 - Code linting

Server Dependencies (in /server)

- express - HTTP server framework
- socket.io - WebSocket server library
- @clerk/backend - Server-side Clerk token verification

3. Database Schema

The database uses PostgreSQL managed through Prisma ORM. The schema is defined in `prisma/schema.prisma` and consists of 5 tables plus 2 enums.

Tables

User (table: users)

Represents a player account. Created automatically when a user signs up via Clerk.

- `id` (String, cuid) - Primary key
- `clerkId` (String, unique) - Links to the Clerk authentication user ID
- `username` (String, unique) - Display name, initially auto-generated as 'avian_xxxx'
- `displayBird` (String, default: 'robin') - Which bird sprite the player has selected
- `createdAt` (DateTime) - When the account was created
- `scores` (Score[]) - All solo race scores for this user
- `matchPlayers` (MatchPlayer[]) - All multiplayer participations

Bot users have `clerkIds` starting with 'bot_' (e.g., 'bot_wpm20'). This convention is used to exclude them from leaderboards.

Passage (table: passages)

A text passage that players type during a race.

- `id` (String, cuid) - Primary key
- `text` (String) - The full passage text to be typed
- `source` (String) - Attribution/origin of the passage
- `wordCount` (Int) - Pre-calculated word count
- `charCount` (Int) - Pre-calculated character count
- `difficulty` (Difficulty enum) - One of: short, medium, long
- `scores` (Score[]) - All scores recorded for this passage
- `matches` (Match[]) - All multiplayer matches using this passage

Score (table: scores)

Records the result of a completed solo race. One per race per user.

- `id` (String, cuid) - Primary key
- `userId` (String) - Foreign key to User
- `passageId` (String) - Foreign key to Passage
- `wpm` (Float) - Words per minute achieved
- `accuracy` (Float) - Accuracy percentage (0-100)
- `ghostData` (Json) - Array of { `charIndex`: number, `ms`: number } for replay
- `createdAt` (DateTime) - When the score was recorded

Indexes: [`passageId`, `wpm` DESC] for fast leaderboard lookups, [`userId`] for player stats.

Match (table: matches)

Represents a multiplayer race room.

- id (String, cuid) - Primary key
- roomCode (String, unique) - The code players use to join (e.g., 'ROBIN-42')
- passageId (String, nullable) - Foreign key to Passage (set when race starts)
- status (MatchStatus enum) - One of: waiting, racing, completed, expired
- startedAt (DateTime, nullable) - When the race began
- createdAt (DateTime) - When the room was created

Index: [status] for finding active rooms.

MatchPlayer (table: match_players)

Join table between Match and User. One entry per player per match.

- id (String, cuid) - Primary key
- matchId (String) - Foreign key to Match
- userId (String) - Foreign key to User
- wpm (Float, nullable) - Final WPM (set when race finishes)
- accuracy (Float, nullable) - Final accuracy
- placement (Int, nullable) - Finishing position (1st, 2nd, etc.)
- ghostData (Json, nullable) - Keystroke replay data
- status (MatchPlayerStatus enum) - One of: racing, finished, abandoned, disconnected
- startedAt (DateTime, nullable) - When this player started typing
- finishedAt (DateTime, nullable) - When this player finished

Unique constraint: [matchId, userId] - prevents duplicate entries.

Enums

- Difficulty: short, medium, long
- MatchStatus: waiting, racing, completed, expired
- MatchPlayerStatus: racing, finished, abandoned, disconnected

Entity Relationships

User 1-to-many Score (a user has many solo race scores). User 1-to-many MatchPlayer (a user can participate in many matches). Passage 1-to-many Score (a passage can have many scores from different users/attempts). Passage 1-to-many Match (a passage can be used in many multiplayer matches). Match 1-to-many MatchPlayer (a match has multiple players). The MatchPlayer table is the many-to-many join between Match and User.

4. Authentication Flow

Authentication is handled by Clerk, a third-party auth provider. Here is the complete flow from sign-up to authenticated gameplay:

Step 1: Sign Up / Sign In

User visits /sign-in or /sign-up. These pages use Clerk's pre-built UI components (<SignIn /> and <SignUp />) with custom styling. The route uses a catch-all pattern [...sign-in]] to handle Clerk's multi-step auth flows (email verification, etc.).

Step 2: Clerk Webhook fires

When a user is created in Clerk, Clerk sends a POST request to /api/webhooks/clerk. This webhook is verified using Svix (Clerk's webhook verification library) by checking the svix-id, svix-timestamp, and svix-signature headers against the CLERK_WEBHOOK_SECRET. On user.created or user.updated, the handler upserts a User record in the database. The username defaults to the Clerk username, or first_name, or an auto-generated 'avian_' + random 8 chars. On user.deleted, the User record is deleted from the database.

Step 3: Middleware checks onboarding status

The Next.js middleware (src/middleware.ts) runs on every request. It uses Clerk's clerkMiddleware() to protect routes. For authenticated users navigating to non-API routes, it calls /api/profile/check?clerkId=xxx. If the response says needsOnboarding: true (meaning the user doesn't exist in the DB yet or still has an auto-generated username starting with 'avian_'), the user is redirected to /onboarding.

Step 4: Onboarding

The /onboarding page (protected route) lets the user choose a username. Validation: 3-20 characters, letters/numbers/underscores only (regex: /^[a-zA-Z0-9_]+\$). On submit, it POSTs to /api/profile/update which upserts the User record. If the username is already taken, Prisma throws a P2002 unique constraint error, which the API returns as a 409 Conflict. On success, the user is redirected to /play.

Step 5: Ongoing Authentication

Protected routes (/profile/*, /onboarding/*) are enforced by the middleware. API routes that require auth (like POST /api/scores) use Clerk's auth() function to extract the userId from the session. The game server also authenticates socket connections by verifying the Clerk JWT token passed in the socket auth payload.

5. Frontend Pages & Routes

/ (Landing Page)

File: `src/app/page.tsx`

The home page. Shows the game title 'Accelerate, Avians' in pixel art style. Displays a menu with three options: SOLO RACE (`/play`), MULTIPLAYER (`/multiplayer`), and LEADERBOARD (`/leaderboard`). Below the menu, it shows a 'Top Racers' preview with the top 5 scores from non-bot users, fetched server-side from the database. Each entry shows rank (with gold/silver/bronze colors for top 3), username, bird icon, and WPM. The page is an async server component.

/sign-in and /sign-up

Files: `src/app/sign-in/[...sign-in]/page.tsx`, `src/app/sign-up/[...sign-up]/page.tsx`

Clerk authentication pages. The `[...sign-in]` catch-all route handles all of Clerk's internal routing for multi-step flows. Styled with custom Clerk appearance config from `src/lib/clerk-appearance.ts`.

/onboarding

File: `src/app/onboarding/page.tsx`

Protected route. Only accessible to authenticated users who haven't set a custom username. Contains a text input for the username and a submit button. Client component that POSTs to `/api/profile/update`. Shows inline error messages for validation failures or duplicate usernames. Redirects to `/play` on success.

/play (Solo Race)

File: `src/app/play/page.tsx`

The main solo gameplay page. Desktop-only (shows `MobileInterstitial` on small screens). This is a client component that uses the `useRace` hook for all game state management.

The flow is:

- 1. SETUP: Player sees difficulty selector (short/medium/long) and clicks 'Start Race'
- 2. LOADING: `useRace` fetches a random passage and ghost opponents from the API
- 3. COUNTDOWN: 3-2-1-GO countdown overlay on the canvas
- 4. RACING: Player types the passage. The canvas shows their bird and ghost birds moving
- 5. FINISHED: Results overlay shows WPM, accuracy, placement vs ghosts

The page renders two main visual components: `RaceCanvas` (the animated bird race) and `TypingArea` (the passage text with cursor). On mount, it fetches the player's profile to get their bird sprite and username.

/multiplayer

File: `src/app/multiplayer/page.tsx`

Entry point for multiplayer. Provides two options: Create a Room or Join a Room (by entering a room code). Links to `/lobby/create` or `/lobby/[code]`.

/lobby/create

File: `src/app/lobby/create/create-room-client.tsx`

Room creation page. Uses the `useMultiplayerRace` hook to emit a `create-room` socket event. On success, redirects to `/lobby/[code]` with the generated room code.

`/lobby/[code]` (Multiplayer Room)

File: `src/app/lobby/[code]/page.tsx`, `lobby-client.tsx`

The dynamic multiplayer room page. Uses `useMultiplayerRace` to manage all room state. Renders one of three sub-components based on the current phase:

- `LobbyWaiting` - Shows player list, share code, difficulty selector (host only), start button (host only)
- `LobbyRace` - The active race. Shows `RaceCanvas` with all players, `TypingArea` for local input
- `LobbyResults` - Final rankings table with placement, WPM, accuracy for each player

`/profile`

File: `src/app/profile/page.tsx`

Player profile page (protected). Shows the player's bird sprite (clickable to open `BirdSelector` modal), stats grid (best WPM, avg accuracy, total races), a WPM history chart (requires 2+ races), a game skin selector, and a race log table showing the last 20 races with date, passage preview, WPM, and accuracy.

`/leaderboard`

File: `src/app/leaderboard/page.tsx`

Global rankings page. Shows top 100 players ranked by WPM. Filterable by difficulty (all, short, medium, long). Table columns: rank, player (username + bird), WPM, accuracy. Excludes bot users. Fetches from `/api/leaderboard` with a 60-second server-side cache.

6. API Routes

GET /api/passages/random

File: src/app/api/passages/random/route.ts

Query params: difficulty (optional, one of: short, medium, long)

Returns a random passage from the database. If difficulty is specified, filters by that difficulty level. Rate limited to 10 requests per 60 seconds per IP address. Returns the full passage object: id, text, source, wordCount, charCount, difficulty.

GET /api/passages/:id/ghosts

File: src/app/api/passages/[id]/ghosts/route.ts

Query params: clerkId (optional), botDifficulty (optional)

Returns up to 4 ghost opponents for a given passage. The logic is:

- 1. If clerkId is provided, look up the user's personal best score on this passage
- 2. Calculate the user's average WPM across all their races (default: 45 WPM for new players)
- 3. Fetch 3 bot ghosts at different skill tiers: one below the user's level, one at their level, one above
- 4. If fewer than 3 bots found at the right tiers, fill remaining slots with any available bots
- 5. Return max 4 ghosts total (personal best + 3 bots)

Each ghost object contains: id, username, displayBird, wpm, ghostData, and isPersonalBest (boolean). Rate limited to 10/60s per IP.

POST /api/scores

File: src/app/api/scores/route.ts

Auth: Required (Clerk). Rate limit: 1 request per 30 seconds per user.

Request body: { passageId, wpm, accuracy, ghostData, totalKeystrokes, correctKeystrokes }

Validation checks:

- ghostData must have at least 10 data points
- WPM must be between 25 and 250 (rejects impossibly slow or fast scores)
- Keystroke counts are checked against the passage character length

Before storing, ghost data is sampled down to a maximum of 500 points using the ghost-sampler utility. Creates a Score record in the database. Returns: { id, wpm, accuracy }.

GET /api/leaderboard

File: src/app/api/leaderboard/route.ts

Query params: difficulty (optional)

Returns the top 100 scores, distinct by userId (only each player's best score). Filters out bot users (where clerkId starts with 'bot_'). Ordered by WPM descending. Results are cached in memory for 60 seconds per difficulty filter to reduce database load. Returns: { entries: [{ userId, username, displayBird, wpm, accuracy, createdAt }] }.

POST /api/profile/update

File: src/app/api/profile/update/route.ts

Auth: Required (Clerk).

Request body: { username? (string), displayBird? (string) }

Upserts the user's profile. Username validation: 3-20 characters, alphanumeric plus underscores only. displayBird must be one of the 19 valid bird types. If the username is already taken, Prisma throws a P2002 unique constraint violation, returned as 409 Conflict. Returns: { user }.

GET /api/profile/:userId

File: src/app/api/profile/[userId]/route.ts

Fetches a user's complete profile stats. The userId param is interpreted as a clerkId. Queries the user record, their last 20 scores (with passage info), and aggregate stats. Returns: { userId, username, displayBird, avgWpm, avgAccuracy, bestWpm, bestAccuracy, totalRaces, recentRaces[] }.

GET /api/profile/check

File: src/app/api/profile/check/route.ts

Query params: clerkId (required)

Used by the middleware to check if a user needs onboarding. Returns { needsOnboarding: true } if the user doesn't exist in the database OR if their username starts with 'avian_' (auto-generated). Otherwise returns { needsOnboarding: false }.

POST /api/webhooks/clerk

File: src/app/api/webhooks/clerk/route.ts

Receives webhook events from Clerk. Verifies the webhook signature using Svix by checking the svix-id, svix-timestamp, and svix-signature headers against the CLERK_WEBHOOK_SECRET environment variable. Handles three event types:

- user.created / user.updated: Upserts a User record with the Clerk user's info
- user.deleted: Deletes the User record from the database

Returns: { received: true } on success.

7. The Typing Engine

File: src/components/typing/typing-engine.ts

The `TypingEngine` class is the core game logic that processes keystrokes and determines if the player is typing correctly. It is NOT a React component - it's a plain TypeScript class that is wrapped by the `useTyping` hook for React integration.

State Properties

- `passage` (string) - The target text the player must type
- `cursorPos` (number) - Current position in the passage (0-indexed)
- `errors` (number) - Count of incorrect keystrokes
- `totalKeystrokes` (number) - Every key press, correct or not
- `correctKeystrokes` (number) - Only correct key presses
- `ghostData` (array) - Array of { `charIndex`, `ms` } recording each correct keystroke
- `isComposing` (boolean) - True during IME composition (for non-Latin keyboards)

Key Methods

`handleKey(key: string, timestampMs: number)`

Called on every keypress. If `isComposing` is true, the key is ignored (IME is handling input). Otherwise, compares the typed key against `passage[cursorPos]`. If it matches: increments `cursorPos`, increments `correctKeystrokes`, and pushes a ghost data point { `charIndex`: `cursorPos`, `ms`: `timestampMs` }. If it doesn't match: increments `errors`. In both cases, increments `totalKeystrokes`. The cursor does NOT advance on errors - you must type the correct character.

`getCurrentWpm(elapsedMs: number): number`

Calculates current words-per-minute. Formula: $(\text{cursorPos} / 5) / (\text{elapsedMs} / 60000)$. The standard typing test convention is that 5 characters equals 1 'word', regardless of actual word boundaries. Returns 0 if elapsed time is 0.

`getAccuracy(): number`

Returns `correctKeystrokes / totalKeystrokes` as a value between 0 and 1. Returns 1 if no keystrokes have been made yet.

`isComplete: boolean (getter)`

Returns true when `cursorPos` equals `passage.length` (player has typed every character).

`reset(newPassage: string)`

Resets all state for a new race with a different passage.

IME Composition Handling

For languages that use Input Method Editors (like Chinese, Japanese, Korean), the engine tracks composition state. During composition (`isComposing = true`), keystrokes are buffered by the browser's IME and not processed by the engine. When composition ends, the final composed character is processed normally.

8. Ghost Replay System

The ghost system is what makes solo races feel competitive. Every completed race records a sequence of keystroke timestamps, and these recordings can be replayed as opponent birds in future races.

How Ghost Data is Recorded

Every time the TypingEngine processes a correct keystroke, it records a data point: { charIndex: number, ms: number }. charIndex is the character position in the passage, and ms is the millisecond timestamp since the race started. A typical race might generate 200-700+ data points depending on passage length.

How Ghost Data is Stored

File: src/lib/ghost-sampler.ts

Before saving to the database, ghost data is downsampled to a maximum of 500 points. The algorithm keeps the first and last data points (to preserve exact start and finish times) and evenly samples the rest. This keeps database storage manageable without noticeably affecting replay quality.

Ghost data is stored as JSON in the Score table (solo races) and MatchPlayer table (multiplayer races).

How Ghost Opponents are Selected

File: src/app/api/passages/[id]/ghosts/route.ts

When a solo race starts, the frontend requests ghosts for the selected passage. The API uses a skill-tier matching system:

- First, it checks for the player's personal best on that specific passage
- Then it calculates the player's average WPM across all their races (defaults to 45 WPM for new players)
- It selects 3 bots at different tiers: one BELOW the player's average, one AT their level, one ABOVE
- This creates a natural difficulty curve - you always have someone to beat and someone pushing you
- Maximum 4 ghosts total (personal best + 3 bots)

How Ghosts are Rendered

During a race, the RaceCanvas interpolates each ghost's position based on elapsed time. It finds the two ghost data points surrounding the current timestamp and linearly interpolates between them to get a smooth bird position. The ghost bird sprites move across the canvas just like the player's bird, creating the feeling of a real race.

Bot Ghost Generation

File: prisma/seed.ts

The seed script generates pre-recorded ghost data for each bot + passage combination. For a bot with target WPM of X, it calculates the expected time per character, then adds realistic jitter (random variation) to each keystroke timestamp so bots don't type with robotic precision. This makes races against bots feel natural.

9. Canvas Rendering

The race visualization is rendered on HTML5 Canvas elements using 2D drawing APIs. No external rendering library is used.

RaceCanvas Component

File: `src/components/race/RaceCanvas.tsx` (342 lines)

This is the main visual component. It manages two canvas elements: a primary canvas for the race and a smaller canvas for the minimap.

Animation Loop

Uses `requestAnimationFrame` for smooth 60fps rendering. The loop updates racer positions, scrolls the camera, draws the parallax background, renders bird sprites, and handles particle effects. All animation state is stored in refs (not React state) to avoid re-renders and stale closure issues.

Parallax Scrolling

File: `src/components/race/parallax.ts`

The background consists of multiple procedurally-generated layers that scroll at different speeds, creating a sense of depth. Closer layers move faster, distant layers move slower. This is a classic 2D game technique.

Camera System

The camera follows the player's bird with momentum and friction. Instead of snapping to the bird's position, it smoothly accelerates toward it and decelerates, creating a polished feel. The camera position determines which part of the race track is visible.

Bird Sprites

File: `src/components/race/bird-sprite.ts`

Each bird has animated sprite frames. The sprite system loads PNG images for each of the 19 bird types and cycles through animation frames during the race. If an image fails to load, a colored rectangle fallback is used.

Confetti Particles

When a race finishes, a burst of confetti particles is spawned at the finish position. Each particle has random velocity, rotation, color, size, and gravity. They animate outward and fade, creating a celebration effect.

Minimap

A small canvas in the corner shows a zoomed-out view of the entire race. Each racer is represented as a colored dot, with a checkered flag at the finish line. Updated at 10Hz (not every frame) to save CPU.

Countdown Overlay

File: `src/components/race/countdown-overlay.ts`

Before the race starts, a '3... 2... 1... GO!' countdown is drawn directly on the canvas with scaling animations.

10. Multiplayer Architecture

Multiplayer is powered by a separate Express + Socket.IO server that handles real-time communication. The server maintains room state in memory (not in the database) during active gameplay, and persists results to the database when races complete.

Server Entry Point

File: server/src/index.ts

Express app with CORS configuration. Socket.IO server listens on port 3001 (configurable). Connection recovery is enabled with a 30-second max disconnect window. On each socket connection, the Clerk auth middleware verifies the token, then room and race event handlers are registered. A /health endpoint is exposed for Railway health checks.

Room Management

File: server/src/room-manager.ts

The RoomManager handles the lifecycle of multiplayer rooms:

- createRoom() - Generates a unique room code, creates a Room object in the in-memory Map
- joinRoom() - Validates the room (exists, not expired, max 6 players, race not in progress), adds player
- removePlayer() - Removes a player. If they were the host, transfers host to the next player. Deletes empty rooms
- disconnectPlayer() - Marks player as offline (allows reconnection within 30-second window)
- reconnectPlayer() - Restores a disconnected player with their new socket ID

Rooms expire after 10 minutes of inactivity. A cleanup loop runs every 60 seconds to remove expired rooms from memory.

Room Codes

File: server/src/room-code.ts

Generates unique 4-6 character alphanumeric room codes in a bird-name format like 'ROBIN-42'. Checks against existing rooms to avoid collisions.

Race Controller

File: server/src/race-controller.ts

Manages the state of an active race. Tracks each player's character index in real time. Calculates WPM, accuracy, and placement as players finish. Handles race timeout - if a configurable duration passes, the race auto-finishes for any players still typing.

Progress Validation

File: server/src/progress-validator.ts

Anti-cheat measure. Validates that each player's reported character index is monotonically increasing (you can't go backward or skip ahead). Also rate-limits progress updates to prevent flooding the server.

Socket Event Flow

The complete multiplayer lifecycle through socket events:

Room Phase:

- Client emits 'create-room' -> Server creates room, joins socket to room channel, emits 'room-created'
- Client emits 'join-room' with code -> Server validates and adds player, emits 'player-joined' to room
- Client emits 'change-difficulty' -> Server updates room difficulty (host only, waiting state only)
- Client emits 'leave-room' -> Server removes player, emits 'player-left', may transfer host

Race Phase:

- Host emits 'start-race' -> Server fetches passage, creates Match + MatchPlayer DB records
- Server emits 'race-countdown' (3, 2, 1) then 'race-started' with passage data
- Players emit 'typing-progress' on each keystroke -> Server validates and broadcasts at ~10Hz
- Player emits 'player-finished' -> Server calculates stats, updates DB, emits 'player-finished' to room
- When all players finish (or timeout) -> Server emits 'race-results' with final rankings

Post-Race:

- Host emits 'play-again' -> Server resets room to waiting state, emits 'room-reset'

Database Persistence

When a race starts, the server creates a Match record (status: racing) and MatchPlayer records for each participant. As players finish, their MatchPlayer records are updated with wpm, accuracy, placement, and ghostData. When the race ends, the Match status is set to 'completed'. This means multiplayer race results are persisted just like solo scores.

11. React Hooks

useRace

File: src/hooks/useRace.ts (206 lines)

The primary hook for solo race gameplay. Manages the complete race lifecycle through these phases: idle -> countdown -> racing -> finished.

What it does:

- Fetches a random passage from /api/passages/random based on selected difficulty
- Fetches ghost opponents from /api/passages/:id/ghosts
- Manages the 3-second countdown timer
- Delegates keystroke handling to useTyping
- Tracks elapsed time during the race
- On completion: submits score to POST /api/scores, calculates placement vs ghost finish times
- Returns everything the UI needs: phase, passage, ghosts, playerProgress, wpm, accuracy, result, handlers

useTyping

File: src/hooks/useTyping.ts (96 lines)

Wraps the TypingEngine class for React. Creates a TypingEngine instance via useRef (so it persists across renders without causing re-renders). Exposes engine state as React-friendly values: cursorPos, hasError, isComplete, wpm, accuracy, ghostData. Provides handleKey and reset callbacks.

useSocket

File: src/hooks/useSocket.ts (104 lines)

Manages the Socket.IO connection to the game server. On mount, initializes a connection using the factory in src/lib/socket.ts, passing the Clerk auth token. Tracks connection state: connecting, connected, disconnected, reconnecting. Uses a singleton pattern - the socket instance persists across page navigations so you don't reconnect every time you move between lobby pages.

useMultiplayerRace

File: src/hooks/useMultiplayerRace.ts (296 lines)

The multiplayer equivalent of useRace. Manages all room and race state through Socket.IO events. Tracks: room code, player list, current phase (waiting/countdown/racing/results), player progresses, rankings, difficulty, host status.

Provides methods:

- createRoom() - Emits create-room event
- joinRoom(code) - Emits join-room event
- startRace() - Emits start-race event (host only)
- sendProgress(charIndex, wpm) - Emits typing-progress
- sendFinished(wpm, accuracy, ghostData) - Emits player-finished
- playAgain() - Emits play-again event (host only)

- `leaveLobby()` - Emits leave-room event
- `changeDifficulty(difficulty)` - Emits change-difficulty event (host only)

12. Rate Limiting & Anti-Cheat

Rate Limiting

File: src/lib/rate-limit.ts (26 lines)

Implements an in-memory sliding window rate limiter. Uses a Map where each key (IP address or user ID) maps to an array of request timestamps. On each request, it filters out timestamps older than the window, then checks if the count exceeds the limit.

Applied to these routes:

- POST /api/scores - 1 request per 30 seconds per user (prevents score spam)
- GET /api/passages/random - 10 requests per 60 seconds per IP
- GET /api/passages/:id/ghosts - 10 requests per 60 seconds per IP

A cleanup interval runs every 5 minutes to purge expired entries from the Map, preventing memory leaks.

Score Validation

File: src/lib/score-validator.ts (47 lines)

When a score is submitted, multiple checks are performed:

- WPM must be between 25 and 250. Below 25 is suspiciously slow (likely AFK). Above 250 is humanly impossible
- Ghost data must contain at least 10 data points (prevents submitting fake minimal data)
- Total keystrokes and correct keystrokes are checked against the passage character length
- The accuracy is recalculated server-side from the raw keystroke counts, not trusted from the client

Multiplayer Anti-Cheat

File: server/src/progress-validator.ts

During multiplayer races, the server validates every progress update from every player:

- Character index must be monotonically increasing (can only go forward, never backward or skip)
- Progress updates are rate-limited to prevent flooding
- Final WPM/accuracy is calculated SERVER-SIDE, not trusted from the client
- Placement rankings are determined by the server based on finish order

13. Seed Script & Bot System

File: prisma/seed.ts (92 lines)

What it does

The seed script populates the database with initial data needed for the game to work. Run via: `npm run db:seed`

- 1. Clears existing passages, scores, match players, matches, and bot users
- 2. Loads passage data from a JSON file (100 passages with balanced difficulty)
- 3. Validates each passage: must be ASCII-only, 30-700 characters in length

- 4. Creates 8 bot users at different WPM levels: 20, 25, 30, 40, 55, 70, 85, 100
- 5. For each bot + passage combination, generates realistic ghost data

Bot Ghost Data Generation

For each bot, the script calculates the expected milliseconds per character based on the bot's target WPM: $\text{msPerChar} = 60000 / (\text{targetWpm} * 5)$. Then for each character in each passage, it generates a timestamp with random jitter added. The jitter makes bot typing feel natural rather than perfectly mechanical. Each bot's ghost data is stored as a Score record linked to the bot User and the Passage.

14. Key Components

RaceCanvas

File: `src/components/race/RaceCanvas.tsx`

Covered in detail in Chapter 9. The main visual canvas for races.

TypingArea

File: `src/components/typing/TypingArea.tsx`

Renders the passage text with a cursor highlight showing the player's current position. Correctly typed characters are styled differently from upcoming characters. Errors are highlighted in red. Shows live stats: WPM, accuracy, and elapsed time. Handles keyboard input events (keydown) and filters out modifier keys (Shift, Ctrl, etc.) and multi-character keys. Supports IME composition events for non-Latin input.

RaceResultsPanel

File: `src/components/RaceResultsPanel.tsx`

Shown after a race completes. Displays final WPM, accuracy, and placement (e.g., '2/4 racers'). Shows whether the score is a personal best. Provides 'Race Again' (same passage) and 'New Race' (different passage) buttons. If the user isn't signed in, shows a prompt to sign in to save scores.

BirdSelector

File: `src/components/BirdSelector.tsx`

A modal grid of all 19 bird species. Each bird is shown as a sprite with its name. Clicking a bird calls the `onSelect` callback and persists the choice to `/api/profile/update`. Available birds include robin, sparrow, cardinal, bluejay, eagle, and many more.

Other Components

- AuthMenu - Sign in/out link in the header, uses Clerk's `useUser` hook
- BirdIcon - Renders a single bird sprite image by bird name
- CrtEffect - CSS overlay that adds retro CRT scanlines across the entire screen
- MobileInterstitial - 'This game is desktop-only' message for mobile/tablet screens
- PixelCrown - Small crown icon rendered next to top-ranked players
- SkinSelector - Lets users choose different visual themes for the game
- WpmChart - Line/bar chart of WPM over recent races (requires 2+ data points)
- AfkBubble - Visual indicator when a player appears to be idle

15. Middleware

Next.js Middleware

File: src/middleware.ts (51 lines)

Runs on every matching request before the route handler. Uses Clerk's clerkMiddleware() as the base. The matcher config excludes static assets, images, and favicon.

Logic flow:

- 1. If the user is authenticated and on an auth page (/sign-in, /sign-up), redirect to /play
- 2. If the user is authenticated and NOT on an API route or /onboarding:
 - a. Call GET /api/profile/check?clerkId=xxx
 - b. If needsOnboarding is true, redirect to /onboarding
- 3. Protected routes (/onboarding/*, /profile/*) require authentication
- 4. All other routes are public

Game Server Auth Middleware

File: server/src/clerk-auth.ts

Socket.IO middleware that runs on every new WebSocket connection. Extracts the Clerk JWT token from socket.handshake.auth.token. Verifies it using Clerk's backend SDK. On success, populates socket.data with userId, username, and displayBird. On failure, disconnects the socket with an authentication error.

16. Environment Variables & Configuration

Required Environment Variables

- DATABASE_URL - PostgreSQL connection string (e.g., postgresql://user:pass@host:5432/db)
- NEXT_PUBLIC_SOCKET_URL - URL of the game server (e.g., https://your-server.railway.app)
- NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY - Clerk frontend publishable key
- CLERK_SECRET_KEY - Clerk backend secret key
- CLERK_WEBHOOK_SECRET - Secret for verifying Clerk webhook signatures
- NEXT_PUBLIC_CLERK_SIGN_IN_URL - Path to sign-in page (/sign-in)
- NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL - Where to redirect after sign-in (/onboarding)

Variables prefixed with NEXT_PUBLIC_ are exposed to the browser. All others are server-only. The DATABASE_URL is used by Prisma to connect to PostgreSQL.

Key Config Files

- next.config.ts - Sentry integration, image optimization disabled, webhook tunnel
- tsconfig.json - TypeScript config with path alias @/* -> ./src/*
- tailwind.config.ts - Custom pixel palette colors, Press Start 2P and Silkscreen fonts
- prisma/schema.prisma - Database schema definition
- vitest.config.ts - Unit test configuration

- playwright.config.ts - E2E test configuration

17. Testing

Unit Tests (Vitest)

Run with: `npm test`. Uses Vitest as the test runner. Tests cover:

- Score validation (WPM bounds, ghost data requirements)
- Ghost data sampling (correct downsampling behavior)
- Rate limiting (window expiry, count enforcement)
- Server-side: Clerk auth middleware, room management, progress validation, race controller, score calculation

End-to-End Tests (Playwright)

Run with: `npm run test:e2e`. Uses Playwright for browser automation. Tests the full user flows in a real browser environment.

18. Deployment

Frontend (Vercel)

The Next.js application deploys automatically to Vercel on git push. Environment variables are configured in the Vercel dashboard. Sentry is integrated for error monitoring. The build command runs `prisma generate` before next build to ensure the Prisma client is generated.

Game Server (Railway)

The Express + Socket.IO server deploys to Railway. Railway provides persistent process hosting (unlike Vercel's serverless model) which is required for WebSocket connections. The server exposes a `/health` endpoint that Railway uses for health checks. CORS is configured to allow requests from the Vercel frontend domain.

Database (Railway)

PostgreSQL is hosted on Railway alongside the game server. Both the Vercel frontend and Railway game server connect to the same database using the `DATABASE_URL` environment variable.

Why Two Servers?

Vercel runs serverless functions that spin up and down per request. WebSocket connections require a persistent, long-running server process. Socket.IO needs to maintain open connections with all players in a room simultaneously for real-time communication. This is fundamentally incompatible with serverless, so the game server runs on Railway which provides traditional always-on server hosting.

19. File Reference Map

A quick reference of every important file and its purpose:

File	Purpose
<code>src/app/page.tsx</code>	Landing page
<code>src/app/play/page.tsx</code>	Solo race page
<code>src/app/multiplayer/page.tsx</code>	Multiplayer entry
<code>src/app/lobby/[code]/page.tsx</code>	Multiplayer room
<code>src/app/lobby/[code]/lobby-client.tsx</code>	Multiplayer room client component
<code>src/app/lobby/create/create-room-client.tsx</code>	Room creation
<code>src/app/onboarding/page.tsx</code>	Username setup
<code>src/app/profile/page.tsx</code>	Player profile
<code>src/app/leaderboard/page.tsx</code>	Global rankings
<code>src/app/sign-in/[...sign-in]/page.tsx</code>	Sign in
<code>src/app/sign-up/[...sign-up]/page.tsx</code>	Sign up
<code>src/app/api/scores/route.ts</code>	Score submission API
<code>src/app/api/leaderboard/route.ts</code>	Leaderboard API
<code>src/app/api/passages/random/route.ts</code>	Random passage API
<code>src/app/api/passages/[id]/ghosts/route.ts</code>	Ghost opponents API
<code>src/app/api/profile/update/route.ts</code>	Profile update API

src/app/api/profile/[userId]/route.ts	Profile fetch API
src/app/api/profile/check/route.ts	Onboarding check API
src/app/api/webhooks/clerk/route.ts	Clerk webhook handler
src/middleware.ts	Auth + onboarding middleware
src/hooks/useRace.ts	Solo race state hook
src/hooks/useTyping.ts	Typing engine hook
src/hooks/useSocket.ts	Socket.IO connection hook
src/hooks/useMultiplayerRace.ts	Multiplayer state hook
src/components/race/RaceCanvas.tsx	Race canvas animation
src/components/race/race-renderer.ts	Canvas 2D drawing
src/components/race/parallax.ts	Parallax background
src/components/race/bird-sprite.ts	Bird sprite animation
src/components/race/countdown-overlay.ts	3-2-1-GO overlay
src/components/typing/TypingArea.tsx	Typing input display
src/components/typing/typing-engine.ts	Core typing logic
src/components/RaceResultsPanel.tsx	Results display
src/components/BirdSelector.tsx	Bird picker modal
src/components/WpmChart.tsx	WPM history chart
src/lib/prisma.ts	Prisma client singleton
src/lib/score-validator.ts	Score validation logic
src/lib/ghost-sampler.ts	Ghost data downsampling
src/lib/rate-limit.ts	Rate limiter
src/lib/socket.ts	Socket.IO client factory
prisma/schema.prisma	Database schema
prisma/seed.ts	Database seed script
server/src/index.ts	Game server entry
server/src/room-manager.ts	Room lifecycle
server/src/race-controller.ts	Race state machine
server/src/progress-validator.ts	Anti-cheat validation
server/src/room-handlers.ts	Room socket events
server/src/race-handlers.ts	Race socket events
server/src/clerk-auth.ts	Server auth middleware
server/src/types.ts	Server type definitions